



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

High-Performance E-commerce Backend Engine

تقرير إنجاز مشروع البرمجة التفرعية في الهندسة المعلوماتية

تقدمة:

حلا سمير أبو علي

بشر ظافر مدور

مالك اسماعيل رستم

حمزة امين محفوض

حسن عدنان شعبان

بإشراف: م. حذيفة عقيل

تعد أنظمة التجارة الإلكترونية الحديثة من أكثر الأنظمة البرمجية تعرضاً للضغط والتعامل المتزامن، وذلك نتيجة الزيادة المستمرة في عدد المستخدمين والطلبات التي تُنفذ في الوقت نفسه. ومع هذا التوسع، أصبحت الحاجة إلى بناء أنظمة خلفية عالية الأداء وقادرة على معالجة العمليات المتوازية بكفاءة أمراً أساسياً لضمان استقرار الخدمة وسلامة البيانات وسرعة الاستجابة. ومن هنا تبرز أهمية مفاهيم البرمجة المتوازية وإدارة التزامن في تصميم الأنظمة الموزعة والأنظمة ذات الأحمال المرتفعة.

يهدف هذا المشروع، المعنون بـ «محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية»، إلى تطوير نظام متكامل يحاكي بيئة تجارة إلكترونية حقيقية قادرة على التعامل مع آلاف العمليات المتزامنة بكفاءة واعتمادية. ويركز المشروع بصورة أساسية على المتطلبات غير الوظيفية المرتبطة بالأداء، وإدارة الموارد، والتحكم في الوصول المتزامن للبيانات، والمعالجة غير المتزامنة، واختبار استقرار النظام تحت الضغط العالي، أكثر من تركيزه على تنوع الواجهات أو الخصائص التقليدية للتطبيقات.

يعتمد المشروع على توظيف مجموعة من المفاهيم المتقدمة في البرمجة المتوازية، مثل حماية البيانات المشتركة من التضارب، واستخدام استراتيجيات الأقفال لضمان سلامة تحديثات المخزون، وتنفيذ المعاملات وفق مبادئ ACID، بالإضافة إلى تطبيق آليات التخزين المؤقت والمعالجة على دفعات وتوزيع الأحمال بين الخوادم. كما يتضمن المشروع قياس أداء النظام وتحليل نقاط الاختناق بهدف تحسين الكفاءة وتقليل زمن الاستجابة.

ومن خلال هذا المشروع، يتم تطبيق المفاهيم النظرية التي تمت دراستها في مقرر البرمجة المتوازية ضمن سيناريو عملي واقعي، يساهم في تنمية القدرة على تصميم أنظمة برمجية مستقرة وقابلة للتوسع، مع تعزيز الحس الهندسي في اختيار الحلول التقنية المناسبة وتبريرها بصورة منهجية.

وصف عام للنظام:

يمثل هذا المشروع منصة تجارة إلكترونية مطورة باستخدام Laravel، تهدف إلى توفير بيئة مستقرة وعالية الأداء لإدارة عمليات الشراء والطلبات ضمن سيناريوهات التشغيل المتزامن والأحمال المرتفعة.

يوفر النظام مجموعة من الخدمات الأساسية التي تشمل إدارة المنتجات والمخزون، معالجة عمليات الشراء والطلبات، إدارة حسابات المستخدمين والمصادقة، بالإضافة إلى إنشاء تقارير وتحليلات للمبيعات بهدف مراقبة أداء النظام وتحليل البيانات.

كما يعتمد النظام على تقنيات المعالجة الخلفية والتخزين المؤقت وآليات التحكم بالتزامن لضمان سلامة البيانات وتحسين سرعة الاستجابة أثناء تنفيذ عدد كبير من العمليات المتزامنة.

التقنيات المستخدمة

تم استخدام Laravel 13 و PHP 8.3 لتطوير الواجهة الخلفية، مع PostgreSQL كقاعدة بيانات رئيسية و Redis للتخزين المؤقت وإدارة الطوابير.

كما تم استخدام Nginx و PHP-FPM لتحسين إدارة الطلبات، بالإضافة إلى Docker و Docker Compose لإدارة وتشغيل الخدمات داخل حاويات مستقلة.

أما بالنسبة لاختبارات الأداء، فقد تم استخدام K6 و JMeter لاختبارات الضغط والتزامن، بالإضافة إلى Telescope و Horizon لمراقبة الأداء وتحليل النظام.

سبب اختيار هذه التقنيات

تم اختيار Laravel لما يوفره من دعم قوي للمعاملات والطوابير وقابلية التوسع، بينما تم اعتماد PostgreSQL لضمان سلامة البيانات والمعاملات.

كما تم استخدام Redis لتحسين الأداء وتسريع الوصول إلى البيانات، و Docker لتسهيل إدارة البيئة وتشغيل الخدمات، بالإضافة إلى K6 لمحاكاة المستخدمين المتزامنين واختبار استقرار النظام.

البنية العامة للنظام

يعتمد النظام على مجموعة من الخدمات المنفصلة التي تعمل معاً لتحقيق الأداء العالي والاستقرار، حيث يتم توجيه الطلبات عبر Nginx إلى Laravel، بينما تتم معالجة المهام الخلفية بشكل مستقل باستخدام Queue Workers.

كما يتم استخدام Redis لإدارة الطوابير والتخزين المؤقت، و PostgreSQL لحفظ البيانات، بالإضافة إلى Horizon و Telescope لمراقبة الأداء وتحليل سلوك النظام أثناء التشغيل.

دورة تنفيذ الطلب داخل النظام

1. إرسال الطلب من المستخدم إلى خادم Nginx.
2. تحويل الطلب إلى Laravel عبر PHP-FPM.
3. معالجة الطلب داخل Controllers والخدمات البرمجية.
4. تنفيذ الاستعلامات على PostgreSQL.
5. إرسال المهام الخلفية إلى Redis Queue عند الحاجة.
6. إعادة الاستجابة للمستخدم.
7. معالجة المهام الخلفية بشكل غير متزامن بواسطة الـ Workers.

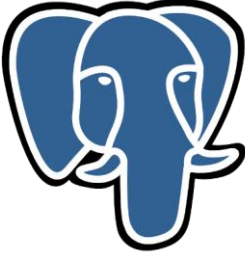
ميزات مراقبة الأداء

يوفر النظام أدوات متعددة لمراقبة وتحليل الأداء، من أهمها Laravel Horizon لمراقبة الطوابير والمهام الخلفية، و Laravel Telescope لتحليل الطلبات والاستعلامات والأخطاء، بالإضافة إلى K6 و JMeter لاختبارات الضغط والتزامن.

NGINX


docker

 redis


PostgreSQL

 k6

المتطلبات الوظيفية:

يتضمن النظام مجموعة من الوظائف الأساسية التي تتيح إدارة عمليات التجارة الإلكترونية وتنفيذها بشكل متكامل، وتشمل ما يلي:

1. إدارة المنتجات:

يُتيح النظام عرض المنتجات وإدارتها مع إمكانية استعراض تفاصيل كل منتج والكميات المتوفرة ضمن المخزون.

2. إدارة سلة التسوق:

يوفر النظام إمكانية إضافة المنتجات إلى سلة التسوق وتعديل الكميات أو إزالة المنتجات قبل إتمام عملية الشراء.

3. إدارة الطلبات وعمليات الشراء:

يسمح النظام للمستخدمين بتنفيذ عمليات الشراء وإنشاء الطلبات مع تتبع حالة الطلبات وسجل العمليات السابقة.

4. التقارير وتحليل المبيعات:

يدعم النظام إنشاء تقارير وتحليلات خاصة بالمبيعات بهدف مراقبة الأداء وتحليل البيانات بشكل دوري.

5. إدارة المستخدمين والمصادقة.



المتطلبات غير الوظيفية:

حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity):

عند تنفيذ عمليات شراء متزامنة على نفس المنتج، ظهرت مشكلة تضارب البيانات (Race Condition)، حيث كانت عدة طلبات تقرأ نفس قيمة المخزون قبل تحديثها، مما يؤدي إلى خصم كميات غير صحيحة أو ظهور قيم سالبة في قاعدة البيانات.

مثلاً لدينا قطعتين من المنتج ذو المعرف 1 باستخدام اختبار K6 يظهر لنا ان النظام يقبل عدد طلبات أكبر من عدد القطع (5 طلبات في هذا المثال) مما يؤدي ظهور القيم المغلوطة.

```
1 export default function (data) {
2   const token = data.tokens[___VU % data.tokens.length];
3
4   const headers = {
5     'Content-Type': 'application/json',
6     'Authorization': `Bearer ${token}`,
7     'Accept': 'application/json',
8   };
9
10  //! STEP 1: Add item to cart
11  const addRes = http.post(
12    `${BASE_URL}/cart/items`,
13    JSON.stringify({
14      product_id: PRODUCT_ID,
15      quantity: QUANTITY,
16    }),
17    { headers }
18  );
19
20  check(addRes, {
21    'add item ok': (r) => r.status === 200 || r.status === 201,
22  });
23
24  //! STEP 2: Checkout immediately (critical race point)
25  const checkoutRes = http.post(
26    `${BASE_URL}/cart/checkout`,
27    JSON.stringify({}),
28    { headers }
29  );
30
31  check(checkoutRes, {
32    'checkout handled': (r) =>
33      r.status === 200 ||
34      r.status === 201 ||
35      r.status === 400 ||
36      r.status === 409,
37  });
38 }
39 }
```

id	name	stock	price	created_at	updated_at
1	corporis	-3	445.93	2026-04-30 20:28:41	2026-05-02 00:01:10
2	id	57	288.87	2026-04-30 20:28:41	2026-04-30 20:28:41
3	sint	47	19.93	2026-04-30 20:28:41	2026-04-30 20:28:41
4	ipsum	33	688.27	2026-04-30 20:28:41	2026-04-30 20:28:41

```
■ TOTAL RESULTS

checks_total.....: 119      7.21511/s
checks_succeeded...: 55.46% 66 out of 119
checks_failed.....: 44.53% 53 out of 119

✓ login success
✓ add item ok
✗ checkout handled
  4  8% - ✓ 5 / ✗ 53
```

الحلول المطروحة:

1. الاعتماد على التحقق البرمجي فقط داخل التطبيق.
2. استخدام Optimistic Locking.
3. استخدام Pessimistic Locking مع Transactions.

الحل المعتمد:

تم اختيار Pessimistic Locking باستخدام lockForUpdate() مع Database Transactions.

```
TOTAL RESULTS

checks_total.....: 137    13.205199/s
checks_succeeded....: 52.55% 72 out of 137
checks_failed.....: 47.44% 65 out of 137

✓ login success
✓ add item ok
✗ checkout handled
  4 2% - ✓ 2 / ✗ 65
```

id	name	stock	price	created_at	updated_at
1	corporis	0	445.93	2026-04-30 20:28:41	2026-05-02 00:01:10
2	id	57	288.87	2026-04-30 20:28:41	2026-04-30 20:28:41
3	sint	47	19.93	2026-04-30 20:28:41	2026-04-30 20:28:41
4	ipsum	33	688.27	2026-04-30 20:28:41	2026-04-30 20:28:41

```
1 public function checkout(User $user)
2 {
3     try {
4         $cart = $user->cart()->firstOrFail();
5         $items = $cart->items()->get();
6
7         if ($items->isEmpty()) {
8             throw new Exception('Cart is empty');
9         }
10
11         $productIds = $items->pluck('product_id');
12
13         $products = Product::whereIn('id', $productIds)
14             ->get()
15             ->keyBy('id');
16
17         foreach ($items as $item) {
18             $product = $products[$item->product_id] ?? null;
19
20             if (!$product || $product->stock < $item->quantity) {
21                 throw new Exception("Out of stock");
22             }
23             sleep(4);
24             $product->decrement('stock', $item->quantity);
25         }
26
27         $order = $user->orders()->create();
28
29         $order->items()->createMany(
30             $items->map(fn($item) => [
31                 'product_id' => $item->product_id,
32                 'quantity' => $item->quantity,
33                 'price_snapshot' => $products[$item->product_id]->price,
34             ]->toArray()
35         );
36
37         $cart->items()->delete();
38         $cart->update(['total_price' => 0]);
39     } catch (Exception $e) {
40         throw $e;
41     }
42 }
43 }
```

```
1 public function checkout(User $user)
2 {
3     DB::beginTransaction();
4
5     try {
6         $cart = $user->cart()->firstOrFail();
7         $items = $cart->items()->get();
8
9         if ($items->isEmpty()) {
10             throw new Exception('Cart is empty');
11         }
12
13         $productIds = $items->pluck('product_id');
14
15         $products = Product::whereIn('id', $productIds)
16             ->lockForUpdate()
17             ->get()
18             ->keyBy('id');
19
20         foreach ($items as $item) {
21             $product = $products[$item->product_id] ?? null;
22
23             if (!$product || $product->stock < $item->quantity) {
24                 throw new Exception("Out of stock");
25             }
26             sleep(4);
27             $product->decrement('stock', $item->quantity);
28         }
29
30         $order = $user->orders()->create();
31
32         $order->items()->createMany(
33             $items->map(fn($item) => [
34                 'product_id' => $item->product_id,
35                 'quantity' => $item->quantity,
36                 'price_snapshot' => $products[$item->product_id]->price,
37             ]->toArray()
38         );
39
40         $cart->items()->delete();
41         $cart->update(['total_price' => 0]);
42
43         DB::commit();
44     } catch (Exception $e) {
45         DB::rollBack();
46         throw $e;
47     }
48 }
```

سبب اختيار الحل:

لأن النظام يتعامل مع بيانات حساسة للغاية التنافس مثل المخزون، وكان المطلوب ضمان منع أي عملية أخرى من تعديل نفس السجل أثناء تنفيذ الطلب الحالي.
يتميز هذا الحل بأنه:

يمنع التضارب بشكل مباشر ويضمن سلامة البيانات حتى تحت الضغط العالي كما يحقق مبدأ ACID في المعاملات.

كما تمت إضافة CHECK Constraint داخل قاعدة البيانات لضمان عدم نزول قيمة المخزون إلى أقل من الصفر حتى في حال حدوث خطأ برمجي مستقبلاً.

```
1 public function up(): void
2 {
3     Schema::create('products', function (Blueprint $table) {
4         $table->id();
5         $table->string('name');
6         $table->integer('stock');
7         $table->decimal('price', 10, 2);
8         $table->timestamps();
9     });
10
11     DB::statement("
12         ALTER TABLE products
13         ADD CONSTRAINT check_stock_non_negative
14         CHECK (stock >= 0)
15     ");
16 }
17
18 /**
19  * Reverse the migrations.
20  */
21 public function down(): void
22 {
23     Schema::dropIfExists('products');
24     DB::statement("
25         ALTER TABLE products
26         DROP CONSTRAINT check_stock_non_negative
27     ");
28 }
```



error: new row for relation "products" violates check constraint "check_stock_non_negative" ^ x

2. إدارة الموارد الحاسوبية (Resource Management & Capacity Control))

المشكلة

عند تشغيل النظام تحت ضغط مرتفع ووجود عدد كبير من المستخدمين المتزامنين، ظهرت مشاكل واضحة تتعلق بارتفاع استهلاك المعالج والذاكرة، إضافة إلى تراجع استقرار النظام وارتفاع زمن الاستجابة. لذلك كان من الضروري تطبيق آليات فعالة لإدارة الموارد والتحكم بعدد العمليات المتزامنة بما يضمن استقرار النظام وعدم انهياره أثناء الضغط العالي.

الحلول المطروحة:

1. زيادة موارد الخادم فقط (Vertical Scaling).
2. تشغيل جميع الخدمات ضمن حاوية واحدة.
3. فصل الخدمات وضبط عدد العمليات والخيط بشكل مدروس.

الحل المعتمد:

تم اعتماد:

1. Docker لفصل الخدمات.
2. ضبط إعدادات PHP-FPM لإدارة العمليات.
3. استخدام Rate Limiting في Nginx.
4. تخصيص Workers مستقلة للمهام الخلفية.

سبب اختيار الحل:

لأن هذا الأسلوب:

- يمنع انهيار الخادم تحت الضغط.
- يسمح بالتحكم الدقيق باستهلاك الموارد.
- يسهل مراقبة الأداء وتحديد الاختناقات.
- أكثر قابلية للتوسع مقارنة بزيادة الموارد بشكل عشوائي.

```
1 worker:
2   image: parallel-fpm
3   container_name: worker
4   command: php artisan queue:work --queue=default,notifications --tries=1
5   environment:
6     DB_CONNECTION: pgsql
7     DB_HOST: postgres
8     DB_PORT: 5432
9     DB_DATABASE: ${DB_DATABASE}
10    DB_USERNAME: ${DB_USERNAME}
11    DB_PASSWORD: ${DB_PASSWORD:-change-me}
12   env_file:
13     - .env.docker
14   volumes:
15     - storage_data:/var/www/html/storage
16   depends_on:
17     postgres:
18       condition: service_healthy
19     redis:
20       condition: service_healthy
21   restart: unless-stopped
22
```

كل

```
1 FROM php:8.3-fpm
2
3 RUN apt-get update && apt-get install -y \
4     git \
5     unzip \
6     libpq-dev \
7     && docker-php-ext-install pdo_pgsql pcntl \
8     && pecl install redis \
9     && docker-php-ext-enable redis \
10    && rm -rf /var/lib/apt/lists/*
11
12 COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
13
14 WORKDIR /var/www/html
15
16 COPY composer.json composer.lock ./
17 RUN composer install --prefer-dist --optimize-autoloader --no-interaction --no-progress --no-scripts
18
19 COPY . .
20
21 COPY .env.docker .env
22
23 RUN composer dump-autoload --optimize --no-interaction
24
25 RUN chown -R www-data:www-data storage bootstrap/cache \
26     && chmod -R ug+rw storage bootstrap/cache
27
28 EXPOSE 9000
29
30 CMD ["php-fpm"]
```

المرحلة الأولى: قبل إجراء أي تحسينات

في البداية تم تشغيل النظام بالإعدادات الافتراضية دون أي ضبط خاص لإدارة العمليات أو التحكم بالطلبات.

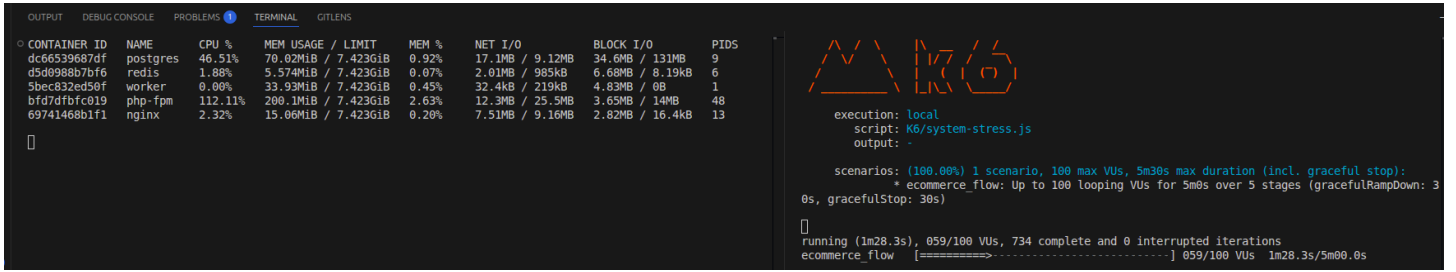
النتائج

أظهرت مراقبة الحاويات أثناء اختبار الضغط:

- ارتفاعاً كبيراً في استهلاك PHP-FPM للمعالج.
- ضغطاً عالياً على الخادم أثناء معالجة الطلبات.
- زيادة ملحوظة في زمن الاستجابة.
- عدم وجود آلية للتحكم بمعدل الطلبات.

كما أظهرت نتائج K6 وجود تباين كبير في الأداء أثناء ازدياد عدد المستخدمين المتزامنين.

الشكل (8): نتائج استهلاك الموارد قبل تطبيق أي تحسينات.



المرحلة الثانية: تحسين إدارة العمليات باستخدام PHP-FPM

بعد ذلك تم ضبط إعدادات PHP-FPM للتحكم بعدد العمليات وطريقة إدارتها، وذلك بهدف:

- منع إنشاء عدد مفرط من العمليات.
- تقليل استهلاك المعالج والذاكرة.
- تحسين استقرار التطبيق أثناء الضغط.

الإعدادات المطبقة

تم تعديل:

- عدد العمليات القصوى pm.max_children
- عدد العمليات الابتدائية pm.start_servers
- عدد العمليات الاحتياطية pm.min_spare_servers
- عدد العمليات الاحتياطية العظمى pm.max_spare_servers

النتائج

بعد تطبيق إعدادات PHP-FPM لوحظ:

- انخفاض واضح في استهلاك الموارد.
- تحسين استقرار الخدمة.
- انخفاض عدد الاختناقات أثناء تنفيذ الطلبات المتزامنة.
- استمرار النظام بالعمل تحت الضغط دون انهيار.

كما أظهرت نتائج docker stats انخفاضاً في استهلاك المعالج مقارنة بالحالة الأولى.

الشكل (9): نتائج استهلاك الموارد بعد ضبط إعدادات PHP-FPM.

```
1 FROM php:8.3-fpm
2
3 RUN apt-get update && apt-get install -y \
4     git \
5     unzip \
6     libpq-dev \
7     && docker-php-ext-install pdo_pgsql pcntl \
8     && pecl install redis \
9     && docker-php-ext-enable redis \
10    && rm -rf /var/lib/apt/lists/*
11
12 COPY docker/php/www.conf /usr/local/etc/php-fpm.d/www.conf
13 COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
14
15 WORKDIR /var/www/html
16
17 COPY composer.json composer.lock ./
18 RUN composer install --prefer-dist --optimize-autoloader --no-interaction --no-progress --no-scripts
19
20 COPY . .
21
22 COPY .env.docker .env
23
24 RUN composer dump-autoload --optimize --no-interaction
25
26 RUN chown -R www-data:www-data storage bootstrap/cache \
27     && chmod -R ug+rxw storage bootstrap/cache
28
29 EXPOSE 9000
30
31 CMD ["php-fpm"]
```

```
1 [www]
2
3 user = www-data
4 group = www-data
5
6 listen = 9000
7
8 pm = dynamic
9
10 pm.max_children = 20
11 pm.start_servers = 4
12 pm.min_spare_servers = 2
13 pm.max_spare_servers = 6
14
15 pm.max_requests = 500
16
17 clear_env = no
```

```
✓ products loaded
✓ product details loaded

HTTP
http_req_duration.....: avg=3.45s min=1.01s med=3.65s max=5.34s p(90)=4.95s p(95)=5.12s
{ expected_response:true }.....: avg=3.45s min=1.01s med=3.65s max=5.34s p(90)=4.95s p(95)=5.12s
http_req_failed.....: 0.00% 0 out of 5443
http_reqs.....: 5443 18.58423/s

EXECUTION
iteration duration.....: avg=11.38s min=4.06s med=12.15s max=16.82s p(90)=15.67s p(95)=15.8s
iterations.....: 1794 6.125318/s
vus.....: 43 min=10 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data received.....: 16 MB 54 kB/s
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
f9c2de30553b	redis	0.69%	5.359MiB / 7.423GiB	0.07%	3.98MB / 1.92MB	5.75MB / 0B	6
f80544b0bfe3	postgres	13.56%	38.31MiB / 7.423GiB	0.50%	33.2MB / 17.7MB	17.9MB / 93.9MB	6
23d971f1f765	php-fpm	39.96%	116.8MiB / 7.423GiB	1.54%	24MB / 50MB	5.4MB / 52MB	21
1d1f9eb21a0b	worker	0.08%	33.91MiB / 7.423GiB	0.45%	68.3kB / 528kB	3.09MB / 0B	1
eb20e64f627f	nginx	0.67%	15.4MiB / 7.423GiB	0.20%	14.7MB / 17.9MB	1.93MB / 16.4kB	13

execution: local
script: K6/system-stress.js
output: -

scenarios: (100.00%) 1 scenario, 100 max VUs, 5m30s max duration (incl. graceful stop):
* ecommerce_flow: Up to 100 looping VUs for 5m0s over 5 stages (gracefulRampDown: 30s, gracefulStop: 30s)

running (4m00.5s), 100/100 VUs, 1443 complete and 0 interrupted iterations
ecommerce_flow [=====] 100/100 VUs 4m00.5s/5m00.0s

المرحلة الثالثة: تطبيق Rate Limiting باستخدام Nginx

في المرحلة الأخيرة تم تفعيل Rate Limiting داخل Nginx للتحكم بعدد الطلبات المرسلة إلى واجهات API خلال فترة زمنية محددة.

الهدف من التطبيق

1. منع إغراق الخادم بعدد هائل من الطلبات.
2. حماية النظام أثناء الضغط المرتفع.
3. تحسين استقرار الخدمات.
4. توزيع الحمل بشكل أكثر توازناً.

```
1 worker_processes auto;
2
3 events {
4     worker_connections 1024;
5 }
6
7 http {
8     include        /etc/nginx/mime.types;
9     default_type    application/octet-stream;
10
11     sendfile on;
12     tcp_nopush on;
13     keepalive_timeout 65;
14
15     limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
16
17     include /etc/nginx/conf.d/*.conf;
18 }
```

```
1     location /api/ {
2         limit_req zone=api_limit;
3         try_files $uri $uri/ /index.php?$query_string;
4     }
```

```
HTTP
http_req_duration.....: avg=60.73ms min=48.84µs med=316.09µs max=3.03s p(90)=2.28ms p(95)=1.01s
{ expected response:true }...: avg=1.03s min=1.01s med=1.02s max=3.03s p(90)=1.05s p(95)=1.08s
http_req_failed.....: 94.21% 48727 out of 51720
http_reqs.....: 51720 170.447815/s

EXECUTION
iteration_duration.....: avg=1.18s min=1s med=1s max=6.56s p(90)=2.02s p(95)=2.09s
iterations.....: 17240 56.815938/s
vus.....: 1 min=1 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 35 MB 114 kB/s
data_sent.....: 6.2 MB 20 kB/s
```

```
2026/05/02 11:00:29 [error] 37#37: *8 limiting requests, excess: 30.080 by zone "api_limit", client: 172.20.0.1, server: _, requ
nginx | 2026/05/02 11:00:29 [error] 36#36: *7 limiting requests, excess: 30.080 by zone "api_limit", client: 172.20.0.1, server: _, requ
nginx | 172.20.0.1 - - [02/May/2026:11:00:29 +0000] "POST /api/login HTTP/1.1" 429 169 "-" "Grafana k6/1.5.0"
nginx | 172.20.0.1 - - [02/May/2026:11:00:29 +0000] "POST /api/login HTTP/1.1" 429 169 "-" "Grafana k6/1.5.0"
nginx | 172.20.0.1 - - [02/May/2026:11:00:29 +0000] "GET /api/products HTTP/1.1" 429 169 "-" "Grafana k6/1.5.0"
nginx | 172.20.0.1 - - [02/May/2026:11:00:29 +0000] "GET /api/products HTTP/1.1" 429 169 "-" "Grafana k6/1.5.0"
nginx | 2026/05/02 11:00:29 [error] 36#36: *7 limiting requests, excess: 30.070 by zone "api_limit", client: 172.20.0.1, server: _, requ
nginx | 2026/05/02 11:00:29 [error] 37#37: *8 limiting requests, excess: 30.070 by zone "api_limit", client: 172.20.0.1, server: _, requ
```

النتائج

بعد تطبيق Rate Limiting:

1. انخفض الضغط على PHP-FPM بشكل واضح.
2. أصبحت الخدمات أكثر استقراراً أثناء اختبار الضغط.

3. تم تقليل عدد الطلبات التي تتسبب باستهلاك مفرط للموارد.

4. تحسن زمن الاستجابة لمعظم الطلبات المقبولة.

لكن بالمقابل ظهرت نسبة من الطلبات الفاشلة بسبب تجاوز الحد المسموح للطلبات، حيث قام Nginx بإرجاع رمز الخطأ 429 Too Many Requests لحماية النظام من الضغط الزائد.

أثبت هذا السلوك نجاح آلية التحكم بالحمل ومنع انهيار الخادم أثناء الذروة.

الشكل (10): نتائج استهلاك الموارد بعد تطبيق Rate Limiting.

الشكل (11): نتائج اختبار الضغط بعد تفعيل التحكم بمعدل الطلبات [OBJ]: الشكل (11): نتائج اختبار الضغط بعد تفعيل التحكم بمعدل الطلبات.

3. المعالجة غير المتزامنة (Asynchronous Processing)

واجه النظام ارتفاعاً في زمن الاستجابة بسبب تنفيذ المهام الثانوية

(مثل إرسال الإشعارات) ضمن المسار الرئيسي للطلب.

الحلول المطروحة:

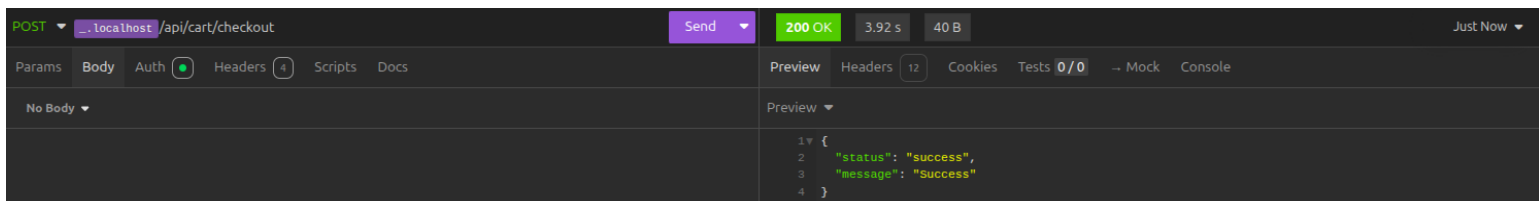
1. تنفيذ جميع العمليات بشكل متزامن.
2. استخدام Threads داخل التطبيق.
3. الاعتماد على Queues لمعالجة المهام الخلفية.

الحل المعتمد:

تم استخدام:

1. Laravel Queues
2. Queue Workers
3. Redis كوسيط لإدارة الطوابير.

```
1 <?php
2
3 namespace App\Notifications;
4
5 use Illuminate\Bus\Queueable;
6 use Illuminate\Contracts\Queue\ShouldQueue;
7 use Illuminate\Notifications\Messages\MailMessage;
8 use Illuminate\Notifications\Notification;
9
10 class OrderCompletedNotification extends Notification
11 {
12     use Queueable;
13
14     /**
15      * Create a new notification instance.
16      */
17     public function __construct()
18     {
19         //
20     }
21
22     /**
23      * Get the notification's delivery channels.
24      *
25      * @return array<int, string>
26      */
27     public function via(object $notifiable): array
28     {
29         return ['database', 'mail'];
30     }
31
32     /**
33      * Get the mail representation of the notification.
34      */
35     public function toMail(object $notifiable): MailMessage
36     {
37         return (new MailMessage)
38             ->line('Your order has been completed.')
39             ->line('Thank you for shopping with us!');
40     }
41
42     /**
43      * Get the array representation of the notification.
44      *
45      * @return array<string, mixed>
46      */
47     public function toArray(object $notifiable): array
48     {
49         return [
50             'message' => 'Your order has been completed.'
51         ];
52     }
53 }
54
```

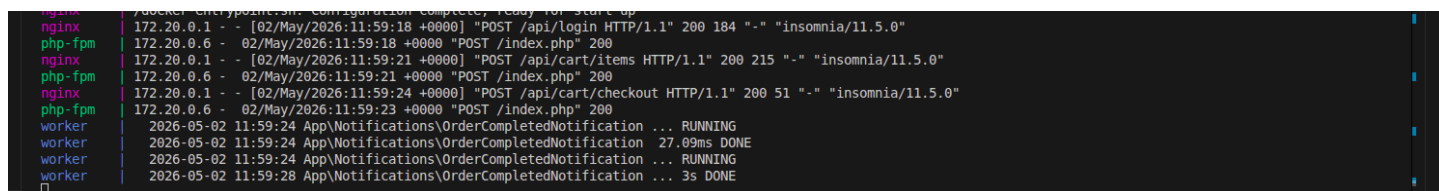
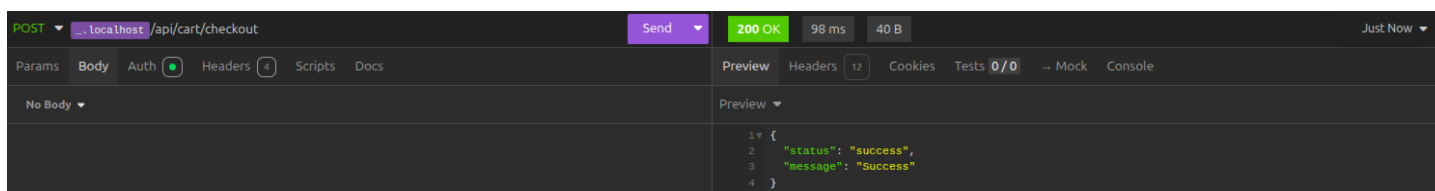


سبب اختيار الحل:

لأن هذا الحل:

1. يفصل العمليات الثانوية عن الطلب الرئيسي.
2. يقلل زمن الاستجابة بشكل كبير.
3. يرفع استقرار النظام أثناء الضغط.
4. يسمح بإعادة تنفيذ المهام الفاشلة بسهولة.

وقد انخفض زمن تنفيذ طلب الشراء من عدة ثوانٍ إلى أجزاء من الثانية بعد تطبيق هذا الأسلوب.



4. معالجة البيانات على دفعات (Batch Processing)

كان المطلوب معالجة كميات كبيرة من البيانات مثل تقارير المبيعات دون التأثير على أداء النظام.

البدائل المطروحة:

1. معالجة جميع البيانات دفعة واحدة.
2. تنفيذ معالجة متزامنة لكل سجل.
3. تقسيم البيانات إلى دفعات (Chunks).

الحل المعتمد:

تم اعتماد المعالجة على شكل دفعات داخل Background Jobs.

سبب اختيار الحل:

لأن معالجة البيانات دفعة واحدة:

1. تستهلك الذاكرة بشكل كبير.
2. تسبب بطئاً في التنفيذ.
3. قد تؤدي إلى توقف العمليات الطويلة.

أما استخدام الـ Chunks:

1. يقلل استهلاك الموارد.
2. يحسن الأداء والاستقرار.
3. يسمح بمعالجة البيانات الضخمة بشكل تدريجي وآمن.

```

1 public function generateProductSalesReports(int $chunkSize = 100): bool
2 {
3     $start = now()->subDay()->startOfDay();
4     $end = now()->endOfDay();
5
6     $query = OrderItem::query()
7         ->join('orders', 'order_items.order_id', '=', 'orders.id')
8         ->whereBetween('orders.created_at', [$start, $end])
9         ->select('order_items.product_id')
10        ->groupBy('order_items.product_id');
11
12    $query
13        ->reorder()
14        ->orderBy('order_items.product_id')
15        ->chunk($chunkSize, function ($rows) use ($start, $end) {
16            $ids = collect($rows)->pluck('product_id')->toArray();
17            ProcessProductSalesReportBatch::dispatch($ids, $start->toDateString(), $end->toDateString());
18        });
19
20    return true;
21 }

```

5. توزيع الأحمال (Load Distribution))

كان من الضروري ضمان استمرار النظام بالعمل بكفاءة أثناء ارتفاع عدد المستخدمين والطلبات المتزامنة، خاصة في بيئة التجارة الإلكترونية التي تتطلب استجابة مستقرة وسريعة.

البدائل المطروحة

1. تشغيل النظام على خادم واحد فقط.

2. زيادة مواصفات الخادم (Vertical Scaling).
3. توزيع الطلبات على عدة خدمات وخوادم (Horizontal Scaling).

الحل المعتمد

تم اعتماد بنية تدعم توزيع الأحمال بين عدة حاويات (Containers) لخدمة PHP-FPM باستخدام Nginx كـ Load Balancer، بحيث يتم توزيع الطلبات الواردة على أكثر من نسخة من خدمة التطبيق.

يعتمد النظام على:

1. تشغيل عدة نسخ من خدمة php-fpm.
2. استخدام upstream داخل Nginx لتوزيع الطلبات تلقائيًا.
3. دعم Docker Compose لعملية التوسع الأفقي عبر الأمر:

```
> docker compose up -d --build --scale php-fpm=4
```

كما تم إعداد Nginx بالشكل التالي:

```
1 upstream php_fpm {
2     zone php_fpm 64k;
3
4     server ${FPM_HOST}:${FPM_PORT} resolve;
5
6     keepalive 32;
7 }
8
```

طريقة توزيع الأحمال المستخدمة

يعتمد Nginx افتراضيًا على خوارزمية **Round Robin** في توزيع الطلبات، وهي الطريقة المستخدمة في هذا النظام.

تقوم هذه الآلية بتوزيع الطلبات بالتتابع بين جميع حاويات php-fpm المتاحة بشكل دوري ومتوازن، بحيث يحصل كل خادم على جزء متقارب من الطلبات.

مثال:

1. الطلب الأول → الحاوية الأولى.
2. الطلب الثاني → الحاوية الثانية.
3. الطلب الثالث → الحاوية الثالثة.
4. ثم يعاد التوزيع من البداية بشكل دائري.


```
> docker compose exec -T php-fpm sh -lc 'grep -F "Request handled by php-fpm" /var/www/html/storage/logs/laravel.log | tail -n 20'
[2026-05-18 19:17:13] local.INFO: Request handled by php-fpm {"fpm_hostname":"bcb6c8511a0e","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"74c528a56e45","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"3eb64e494e58","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"15b6a43bdff7","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"bcb6c8511a0e","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"74c528a56e45","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"3eb64e494e58","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"15b6a43bdff7","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"bcb6c8511a0e","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"74c528a56e45","method":"GET","path":"/","fpm_upstream":""}
[2026-05-18 19:17:14] local.INFO: Request handled by php-fpm {"fpm_hostname":"3eb64e494e58","method":"GET","path":"/","fpm_upstream":""}
```

يساعد ذلك على:

1. تحقيق توازن عادل في استهلاك الموارد.
2. تقليل الضغط على خادم واحد.
3. تحسين زمن الاستجابة عند زيادة عدد المستخدمين.
4. رفع كفاءة المعالجة أثناء اختبارات الضغط (Stress Testing).

دعم الاستقرار أثناء الضغط

تم أيضًا استخدام آلية Rate Limiting داخل Nginx لحماية النظام أثناء اختبارات الضغط ومنع الاستهلاك المفرط للموارد، خصوصًا على واجهات API الحساسة مثل:

1. تسجيل الدخول.
2. إضافة المنتجات إلى السلة.
3. عمليات الدفع والطلبات.

مثال:

```
1 # Login
2 location /api/login {
3     limit_req zone=api limit burst=15 nodelay;
4     try_files $uri $uri/ /index.php?$query_string;
5 }
6
7 # Product listing
8 location /api/products {
9     limit_req zone=api limit burst=30 nodelay;
10    try_files $uri $uri/ /index.php?$query_string;
11 }
```

سبب اختيار الحل

تم اختيار توزيع الأحمال للأسباب التالية:

- منع تشكل نقطة اختناق واحدة (Single Point of Bottleneck).
- تحسين استقرار النظام تحت الضغط العالي.
- دعم التوسع المستقبلي بسهولة عبر إضافة حاويات جديدة.
- رفع قدرة النظام على معالجة عدد أكبر من الطلبات المتزامنة.
- تحسين الاستفادة من الموارد وتقليل احتمالية توقف الخدمة.
- تحقيق توازن أفضل بين الأداء وقابلية التوسع (Scalability).